Se usó Cloude Sonnet 5.5 como referencia principal de búsqueda de información para las interfaces gráficas, como los estilizados de los componentes de mui ejemplo (card) esto fue en funcionamiento similar a una búsqueda web o en stackOverflow optimizando tiempos de búsqueda en las vistas de Vista BD, PaseLista, y Escaneo QR el proceso de uso fue primero veo en el mockup como se deberia de ver el componente después le pregunto a la herramienta cloud como se escribe el código de ese componente y ya con eso yo escribo el componente

en la función para el escaneo con qr se utilizó como buscador para ver como se usaba la librería y como llamarla ya con esta información pude hacer la función lector QR que empieza en la línea 51 y acaba la línea 101 que se apoya de la explicación inicial de la IA para ver como la llama once que es el componente de la librería

En el Theme.tsx se le proporcionó a Cloude Sonnet 5.5 ver el mockup realizado e instrucciones de fuentes con estilos para la parte de diseño. Después de eso se fue mejorando a través de diferentes sub prompts para mejorar accesibilidad y lo atractivo visual, y la forma de implementar las fuentes usadas directamente en el theme

En App.tsx se usó para debugger donde se agregó el appcontext de dyslexia para implementar la funcionalidad del cambio de fuente a openDyslexic en todos los campos simultáneamente sin cambios al aria o estilos visuales separados al estilo de texto.

Dentro del index.js en el backend se utilizó claude Sonnet 5.5 para programar cómo podíamos hacer en estadísticas para guardar las diferentes fechas que comparar.
prompt: Necesito que tener fechas para comparar los datos de esas fechas, dame el codigo para poder tener fecha de: hoy,dias del periodo, inicio del periodo e inicio del periodo anterior y explicame como puedo usar estas fechas para sacar las estadísticas de tasa ausencia y asistencia a su vez que la tendencia, no cometas errores.

Dentro del mismo archivo con el mismo modelo de claude para el endpoint de incidencias lo utilizamos para consultar problemas que estábamos teniendo con el manejo de la información, con la corrección de este endpoint pudimos mejorar y tomarlo como referencia para los siguientes endpoints.
Prompt:Estamos teniendo problemas con este endpoint, dime ciertas cosas que podrían hacer que este endpoint cause errores.

A lo largo del proyecto se utilizó la inteligencia artificial de manera ética, entendiendo siempre cómo funciona el output de la IA para aplicarlo a nuestro proyecto, dado que no se utilizó un solo prompt para cada caso. Se desarrollará una explicación de lo que se pidió y se desarrolló con la IA.

Los prompts fueron hechos con Claude Opus 5.5 con effort en high.

Para la base de datos:
Se pidieron sugerencias sobre qué librerías se pueden usar para encriptar los datos con AES-256. Buscando encontrar las ventajas y desventajas de cada una.

Se pidió un script para poder encriptar los datos sensibles de menores que no estén cifrados sin romper el esquema de Mongoose y con opción de encriptar datos existentes e importados.

Se pidió un script con “dummy data” en cada colección para poder probar la base de datos.

Para el manual de instalación:

Se pidió generar una portada basada en el feel de los manuales del ISSSTE.

Se pidió verificar ortografía, redacción y estructura del manual para verificar que el contenido está completo.

En base al mockup realizado del proyecto, Claude me ayudó a averiguar qué elementos de Mui se podían usar para parecerse lo más posible a dichos mockups proporcionados. De esta forma se nos proporcionaron los componentes de Cards (CardContent, CardHeader, CardActionArea), Boxes, Chip, Dialog (DialogActions, DialogContent, DialogTitle), FormControlLabel, Radio (RadioGroup), ToggleButton, ToggleButtonGroup y Tab/Tabs y ya a partir de ahí se aprendió acerca de dichos componentes para ver cómo funcionaban y cómo debería ser la implementación.

Un ejemplo de dichos prompts seria:
“En base al mockup proporcionado, que me recomendarías utilizar para poder hacer [parte de la página]”

En varias partes de mi código se usa “gutterBottom” para dejar un espacio entre el texto y lo que va abajo, yo no lo conocía previamente y fue algo entregado por Claude.
Prompt: “Ayudame a hacer que se vea mas separado de abajo [imagen]”
En todas mis partes se tenía que traducir la fecha que se guardaba en formato ISO en la base al estándar de Mexico. En esta función hubo una ayuda de Claude en la forma de hacerlo con la idea de utilizar “toLocaleDateString” para pasar dicho string a hora tiempo México e “isNan” para la detección si la fecha es inválida. Soy plenamente consciente de que hacen ambas funciones y se investigó cómo funcionaban.
Prompt: “Si estoy recibiendo una fecha en formato ISO, que funciones puedo utilizar para pasar dicha fecha a un horario de México y comprobar que lo que se mando si es una fecha”

Esto se utilizó en las funciones “fechaTexto” y “soloFecha”.

La idea de utilizar SlotProps fue de Claude para poder arreglar problemas con los labels en las secciones de filtros y en la sección de headers.
Prompt: “Como puedo arreglar los problemas de Orphaned Label que me muestra WAVE en las cajas de filtros?”

Dentro de RegistrarLlamadaDialog en la parte del botón “Cerrar” aparecía muy junto el texto con el label de “Esc”. Claude propuso el utilizar “ ” para poder crear un espacio entre ambos elementos.

Prompt: “Como puedo hacer que haya más espacio entre el texto de cerrar y el esc”
En la parte de estadísticas, el convertir un número que viene como null(suprimido) a un <5 o en porcentaje fue algo corregido por Claude ya que mis intentos habían sido equivocados. Claude me ayudó a incluir el === para que funcionara y me dio toda la parte de `${v.toFixed(1)}%` en lo del porcentaje. Soy consciente de que hacen y cómo funcionan
Prompt: “Ayúdame a que esta función const num = (v: Suprimible) => (v == null ? "< 5" : String(v)); funcione y también para un porcentaje”

En Indicador de estadísticas, utilizó component="span", esto fue dado por Claude igual para arreglar un problema con el header.

Prompt: “Ayudame a arreglar el problema de heading [imagen]”
En la parte de estadísticas Claude me ayudó a hacerlo de la forma parecida al mockup con la función de “RejillaIndicadores” para que se acomoden los indicadores principales de la pantalla. Se lo que hace y se como funciona, pero su implementación fue tal cual Claude lo proporcionó puesto que su propuesta funcionó perfecto y se entendió lo que se hacía
Prompt: “Como hago que las pestañas de indicadores se vean bien”
Respuesta: “const RejillaIndicadores = ({ children }: { children: ReactNode }) => (<Box sx={{ display: "grid", gap: 2, gridTemplateColumns: { xs: "1fr 1fr", md: "repeat(4, 1fr)" } }}>{children}</Box>);”

Dentro de estadísticas se necesitaba un tiempo de espera, para esto Claude nos ayudó a implementarlo con un useEffect que daba una pequeña espera para pasar los datos. Igualmente la propuesta de Claude fue eficiente y se dejó, sabiendo plenamente el cómo funciona y qué es lo que hace.
Prompt: “Necesito que haya una pequeña espera cuando se mandan ciertos datos, ayudame”
Respuesta: “ useEffect(() => {
const t = setTimeout(() => setEstancia(busqueda.trim()), 400); // trim quita espacios sobrantes
return () => clearTimeout(t);
}, [busqueda]);”

Dentro de estadísticas, usamos un const consultas para que el handleGetData solo traiga la información con los filtros puestos por el usuario. Esto fue con ayuda de Claude ya que nos dio el cómo debería estar estructurado el handleGetData para que esto se cumpla (la estructura base ya se tenía) y el uso de URLSearchParams. Todo esto se entiende su funcionamiento.

Prompt: “Necesito que el handelgetdata solo me de la información de los filtros, ayúdame a que esto funcione”